

FA23-BCE-113
Muhammad Ahmad
MSI-Lab # 1

Task 1: Fill the table after compiling the code

Write the following program in editor, compile and emulate it. Also fill the table.

		AH	AL	BH	BL	CH	CL	SI	DS
1.	MOV AX,0A9H	0	A9						
2.	MOV BH,36			24	0				
3.	MOV SI,2345H							2345	
4.	MOV BL,24H				24				
5.	MOV AX,BX	24	24						
6.	MOV CX,SI					23	45		
7.	MOV DS,CX								2345
8.	MOV AH,0CDH	CD							
9.	XCHG AH,BH	24		CD					
10.	XCHG CX,BX			23	45	CD	24		
11.	MOV AX,1200H	12	00						
12.	MOV DS,AX								1200
13.	MOV [0004H],78H	12	00	23	45	CD	24	2345	1200

registers		
	H	L
AX	12	00
BX	23	45
CX	CD	24
DX	00	00
CS	0100	
IP	0034	
SS	0100	
SP	FFFE	
BP	0000	
SI	2345	
DI	0000	
DS	1200	
ES	0100	

Description: The table is filled in an incremental way, like at every step, value of registers are updated according to the instruction given, the new value is written to register when it is overwritten by the new register, at the end, all upgraded values are written in one row, also the screenshot of final register content is also attached above

Task 2: Without using XCHG command, write 8086/8088 Assembly language program that swap the contents of

a) SP and DI registers

```
MOV AX, SP
MOV SP, DI
MOV DI, AX
```

Description: SP is the stack pointer, DI is the destination index, by default SP points to the top of the stack (sp = FFFE), We use a helper register AX to store the value of SP, and then update the SP with the value of the DI and at the end we update the DI with the value store in helper register AX

registers		
	H	L
AX	FF	FE
BX	00	00
CX	00	00
DX	00	00
CS	0100	
IP	001A	
SS	0100	
SP	0000	
BP	0000	
SI	0000	
DI	FFFE	
DS	0100	
ES	0100	

b) SS and DS registers

```
MOV BX, DS
MOV AX, SS    ; NOTE: MOV SS,DS , Move DS,SS (WE ARE UNABLE TO DO
              ; THAT          IT CAN BE DONE DURING EXECUTION
MOV SS, BX
;mov bx , ax
MOV DS, AX
```

Description: We are unable to change the content of SS and DS so we first store them in a helper register and then repeat the task 1

registers		
	H	L
AX	01	00
BX	01	00
CX	00	00
DX	00	00
CS	0100	
IP	001C	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0100	
ES	0100	

c) Memory locations ABCDH: 2345H and 1234H: 78DEH

```
; load ABCDH: 2345H into bx
MOV AX , 0abcdh
MOV DS , AX
MOV [2345h],44h ; hamara kaam
MOV BX,[2345h] ; bx = 44h
; load 1234H: 78DEH into cx
MOV AX , 1234h
MOV DS , AX
MOV [78DEH],45h ; hamara kaam
MOV CX , [78DEH] ; 45h
```

```
; swap bx and cx
MOV DX , BX
MOV BX , CX
MOV CX , DX
```

; now store value back to memory

```
MOV AX , 0abcdh
MOV DS , AX
MOV [2345h],BX
```

```
MOV AX , 1234h
MOV DS , AX
MOV [78DEH],CX
```

```
; now load back to verify
; load ABCDH: 2345H into bx
MOV AX , 0abcdh
MOV DS , AX
MOV BX,[2345h]
```

```
; load 1234H: 78DEH into cx
MOV AX , 1234h
MOV DS , AX
MOV CX , [78DEH]
```

registers		
	H	L
AX	12	34
BX	00	44
CX	00	45
DX	00	00
CS	0100	
IP	001C	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	1234	
ES	0100	

registers		
	H	L
AX	12	34
BX	00	45
CX	00	44
DX	00	44
CS	0100	
IP	005A	
SS	0100	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	1234	
ES	0100	

Description: We first store the values at both memory locations ABCDH: 2345H and 1234H: 78DEH also load them in BX , CX respectively . Then we swap BX and CX using the method use in task2a. Finally, we store back the swapped values in the respective address location. To verify

result we load back the values from respective address, or we also see the swapped value from the memory